1. Time and Space Complexity

Complexity estimates how an algorithm's resource use grows with input size n. Big-O describes an upper bound on growth and is commonly used for worst-case analysis in beginner problems.

Common time complexities

Growth	Name	Typical example
O(1)	Constant	Access a[i]
O(log n)	Logarithmic	Binary search in a sorted array
O(n)	Linear	Scan an array once
O(n log n)	Linearithmic	Typical efficient comparison sorts, e.g. merge sort
$O(n^2)$	Quadratic	Compare all pairs / simple nested loops
$O(n^3)$	Cubic	Three nested loops over n
$O(2^n)$	Exponential	Many exhaustive subset-recursion algorithms
O(n!)	Factorial	Enumerating all permutations

For large inputs, the rough order from more scalable to less scalable is O(1), O(log n), O(n), O(n log n), $O(n^2)$, $O(n^3)$, $O(2^n)$, O(n!). Actual performance also depends on constants, memory, input, and implementation.

How to count loops

```
// O(n)
for (int i = 0; i < n; i++) cout << i;

// O(n^2)
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++) cout << i << j;

// O(log n): repeatedly halve a value
while (n > 1) n /= 2;
```

- Sequential work is added; keep the dominant term: $O(n) + O(n^2) = O(n^2)$.
- Nested loops multiply iteration counts when each runs about n times.
- Ignore constants and lower-order terms: O(3n + 10) becomes O(n).
- Don't assume every nested loop is automatically $O(n^2)$; inspect its bounds.

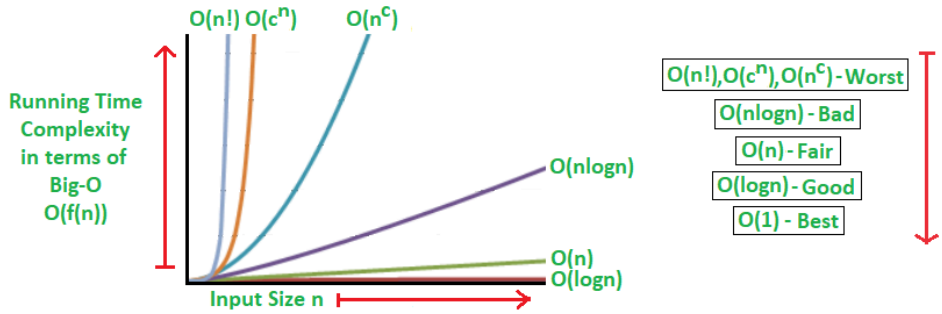
Space complexity

- Auxiliary space is extra memory used by the algorithm, excluding input.
- A fixed number of variables uses O(1) auxiliary space; an extra array of n elements uses O(n).

```
int sum = 0;            // O(1) extra space
int* copy = new int[n]; // O(n) extra space
```

Recursive calls use call-stack space. Recursion depth n commonly means O(n) stack space; balanced divide-and-conquer recursion may use O(log n) stack depth.

In-place algorithms usually use O(1) auxiliary space, though recursion or library implementation details can change the total.



Rules for simplifying complexity

- Best case: the least work for an input of size n.
- Average case: expected work under an assumed input distribution.
- Worst case: the most work for an input of size n.
- Drop constant factors: O(3n) becomes O(n); O(5n²) becomes O(n²).
- Keep the fastest-growing term: O(n² + n + 7) becomes O(n²).
- Sequential parts add: O(n) + O(n²) = O(n²).
- Nested independent loops multiply: n iterations inside n iterations gives O(n²).

2. Two Pointers

Two pointers means using two indices/references to process data efficiently. It can replace nested comparisons with a single pass when pointer movements are logically valid.

Pattern A: pointers moving inward

```
// Reverse an array in-place
int left = 0, right = n - 1;
while (left < right) {
   swap(arr[left], arr[right]);
   left++;
   right--;
}
```

Dry run [1, 2, 3, 4, 5]: swap indices (0,4) → [5,2,3,4,1]; swap (1,3) → [5,4,3,2,1]; stop when left >= right.

Pattern B: same direction / fast and slow

Both pointers move forward, often at different speeds or with different meanings.

```
// Remove duplicates from a sorted vector in-place.
// Returns the length of the unique prefix.
int removeDuplicates(vector<int>& a) {
   if (a.empty()) return 0;

   int slow = 0; // last position containing a unique value
```

```
    for (int fast = 1; fast < a.size(); ++fast) {
        if (a[fast] != a[slow]) {
            ++slow;
            a[slow] = a[fast];
        }
    }
    return slow + 1;
}
```

The unique values occupy a[0] through a[returned_length - 1]. Values beyond that prefix are not meaningful for the returned result. Time O(n), extra space O(1).

Pattern C: pair sum in a sorted array

```
int left = 0, right = n - 1;
while (left < right) {
    int sum = arr[left] + arr[right];
    if (sum == target) {
        cout << "Found";
        break;
    } else if (sum < target) {
        left++;        // Need a larger sum
    } else {
        right--;       // Need a smaller sum
    }
}
```

The pair-sum strategy relies on the array being sorted.

If sum is too small, move left rightward; if too large, move right leftward.

For two distinct positions, stop when left >= right.

3. Two-Dimensional Arrays

3.1 Definition and indexing

A 2D array represents data arranged in rows and columns, such as a matrix or grid. For an array with r rows and c columns, valid row indices are 0 through r - 1 and valid column indices are 0 through c - 1.

```
const int rows = 3, cols = 4;
int matrix[rows][cols] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

cout << matrix[1][2]; // 7: row 1, column 2
```

C++ uses zero-based indexing. matrix[i][j] means row i, column j. Access outside valid bounds causes undefined behavior for built-in arrays.

3.2 Declaration and initialization

```
int a[2][3]; // built-in array; elements are uninitialized if local and not initialized
int b[2][3] = {{1, 2, 3}, {4, 5, 6}};
int c[2][3] = {}; // all elements value-initialized to zero
```

```
vector<vector<int>> grid(3, vector<int>(4, 0)); // dynamic 3 x 4 grid
```

3.3 Traversing a 2D array

```
// Built-in array with known column count
for (int i = 0; i < rows; ++i) {
    for (int j = 0; j < cols; ++j) {
        cout << matrix[i][j] << ' ';
    }
    cout << '\\n';
}

// Dynamic vector-of-vectors
for (int i = 0; i < grid.size(); ++i) {
    for (int j = 0; j < grid[i].size(); ++j) {
        cout << grid[i][j] << ' ';
    }
    cout << endl;
}
```

A full traversal takes O(rc) time. The nested loops do not automatically mean $O(n^2)$; with r rows and c columns, the precise bound is O(rc). If $r = c = n$, this becomes $O(n^2)$.

3.4 Input and output

```
int r, c;
cin >> r >> c;
vector<vector<int>> a(r, vector<int>(c));

for (int i = 0; i < r; ++i) {
    for (int j = 0; j < c; ++j) {
        cin >> a[i][j];
    }
}

for (int i = 0; i < r; ++i) {
    for (int j = 0; j < c; ++j) {
        cout << a[i][j] ;
    }

       cout<<endl;
}
```

3.5 Common matrix operations

Row sums and column sums

```
// Row sums
for (int i = 0; i < r; ++i) {
    long long sum = 0;
    for (int j = 0; j < c; ++j) sum += a[i][j];
    cout << sum << '\\n';
```

```
}

// Column sums
for (int j = 0; j < c; ++j) {
    long long sum = 0;
    for (int i = 0; i < r; ++i) sum += a[i][j];
    cout << sum << ' ';
}
```

Main and secondary diagonals

For an n × n square matrix, the main diagonal has positions (i, i). The secondary diagonal has positions (i, n - 1 - i). These formulas assume a square matrix.

```
long long mainSum = 0, secondarySum = 0;
for (int i = 0; i < n; ++i) {
    mainSum += a[i][i];
    secondarySum += a[i][n - 1 - i];
}
```

For odd n, the center element belongs to both diagonals. If asked for the sum of both diagonals without counting the center twice, subtract the center element once.

Transpose

The transpose swaps rows and columns: an r × c matrix becomes c × r, with transposed[i][j] = original[j][i].

```
vector<vector<int>> transpose(vector<vector<int>>& a) {
    int r = a.size();
    int c =a[0].size();
    vector<vector<int>> t(c, vector<int>(r));

    for (int i = 0; i < r; ++i)
        for (int j = 0; j < c; ++j)
            t[j][i] = a[i][j];

    return t;
}
```

Time O(rc), extra storage O(rc) for the returned matrix. A square matrix can be transposed in place by swapping a[i][j] with a[j][i] only for j > i.

3.6 Boundary and shape patterns

First row: i == 0. Last row: i == r - 1.

First column: j == 0. Last column: j == c - 1.

Boundary cell: i == 0 || i == r - 1 || j == 0 || j == c - 1.

Square matrix: r == c. Do not assume a matrix is square unless stated.

Other Reads:

https://www.geeksforgeeks.org/dsa/calculation-of-address-of-element-of-1-d-2-d-and-3-d-using-row-major-and-column-major-order/

https://www.geeksforgeeks.org/cpp/cpp-matrix-multiplication/

4. Character Arrays

character array is an array used to store multiple characters, such as letters, digits, symbols, or a word.

For example, the word `"HELLO"` contains five characters:

```
char arr[5] = {'H', 'E', 'L', 'L', 'O'};
```

Here:

- `char` is the data type.
- `arr` is the array name.
- `5` is the number of elements.
- Each element stores one character.

Different ways to declare character arrays

There are several ways to create a character array.

Method 1: Declare without initializing

```
char arr[5];
```

This creates space for five characters, but the elements are not automatically initialized to meaningful values in a local variable.

Method 2: Initialize individual characters

```
char arr[5] = {'H', 'E', 'L', 'L', 'O'};
```

This stores exactly five characters. There is no extra space reserved for a null terminator.

Method 3: Initialize using a string literal

```
char arr[] = "HELLO";
```

The array contains six characters in storage:

```
H, E, L, L, O, '\0'
```

The null character tells C++ where the string ends.

Difference between Char Array and String in c++

The below table illustrates the major differences between char array and string in C++.

Feature	Char Array	String
Memory Management	Static - Fixed size upon declaration	Dynamic - Resizes automatically
Null-Termination	Null-terminated ('\0').	Manages length internally and also support style representation with null termination.
Functionality	Requires manual handling of character sequences	Rich set of member functions (e.g., concatenation, substring extraction)
Ease of Use	May be less intuitive, more error-prone.	More intuitive and convenient due to member functions.

Compatibility	Compatible with C-style string functions.	Part of C++ Standard Library, may not be compatible in all environments.
Portability	Compatible with C and C++ both.	Standardized in C++ environments, may not be available in some scenarios like C style environments.

Read about different functions of strings:

https://www.geeksforgeeks.org/cpp/strings-in-cpp/

5. Practice questions

- Why does binary search require sorted data?
- Reverse an array using two pointers.
- Check whether the string 'level' is a palindrome.
- Print every element of a 3 × 2 matrix.
- Find the sum of each row in a matrix.
- Why does char s[] = "cat" need 4 character slots?
- What is the difference between cin >> s and getline(cin, s)?

https://leetcode.com/problems/reverse-string

https://leetcode.com/problems/transpose-matrix

https://leetcode.com/problems/two-sum-ii-input-array-is-sorted

https://leetcode.com/problems/matrix-diagonal-sum

https://leetcode.com/problems/valid-palindrome